

Trabalho 1 PAD

Igor Basilio Valerão

21 de agosto de 2026

1 Introdução

Esse trabalho lida com a implementação de uma solução para o problema de distâncias entre vetores de um mesmo tamanho T e a sua armazenagem em um outro vetor, também conta com a análise dos resultados quando o tamanho dos vetores T aumenta com valores de 32 até 4096.

2 Metodologia

O sistema operacional utilizado foi o windows 11 versão 25H2 26200.9168, o computador utilizado para rodar os experimentos possui as seguintes configurações mostradas abaixo, além disso o laptop rodou os experimentos enquanto estava com bateria cheia e ligado na tomada.

CPU AMD Ryzen 7 5800H with Radeon Graphics (16 CPUs), 3,2GHz

GPU NVIDIA GeForce GTX 1650

RAM 16384 MB DDR4 Ram

SO Sistema Operacional Windows 11

O algoritmo utilizado é o mais lento possível com tempo de execução $\theta(N * T)$, o algoritmo é mostrado no código 1.

```
1 for (int j = 0; j < X.size(); j++)
2 {
3     float sum = 0;
4     for (int k = 0; k < X[j].size(); k++)
5     {
6         sum += pow(q[k] - X[j][k], 2);
7     }
8     res[j] = sum;
9     sum = 0;
10 }
```

Código 1: Algoritmo utilizado para computar as distâncias entre vetores.

onde N é o número de vetores e T o tamanho de cada vetor, instruções SIMD, threads, GPU ou outros aceleradores não foram utilizados, e otimizações do compilador também não foram utilizadas, apenas utilizando otimizações de compiladores o desempenho seria muito melhor já que esse problema pode ser melhorado utilizando instruções SIMD e o compilador utiliza-se destas instruções quando qualquer nível de otimização é utilizado como `-O3`, `-O2` e `-O1`. O número de vetores foi escolhido como $N = 10$, os dados foram coletados passando o tamanho T dos vetores como argumento para o executável por exemplo : `mainNonOptimized 4096` o executável emitia o tempo de execução em milissegundos no `stdout`, esse processo foi repetido 30 vezes para cada tamanho T , os dados estão presentes em google sheets [1] e o código disponível no github [2].

3 Análise dos resultados

Nesta seção é feita uma análise dos resultados retirados do experimento, começando pelo tempo de execução na figura 1 é mostrado o tempo de execução com N fixo em 10 e T variando de tamanho entre 32 até 4096, é possível notar que o tempo de execução dobra com o aumento de 32 para 64 e de 64 para 128 isso se mantém até 4096, com o gráfico é mais notável que existe uma semelhança forte com um aumento exponencial de valor 2.

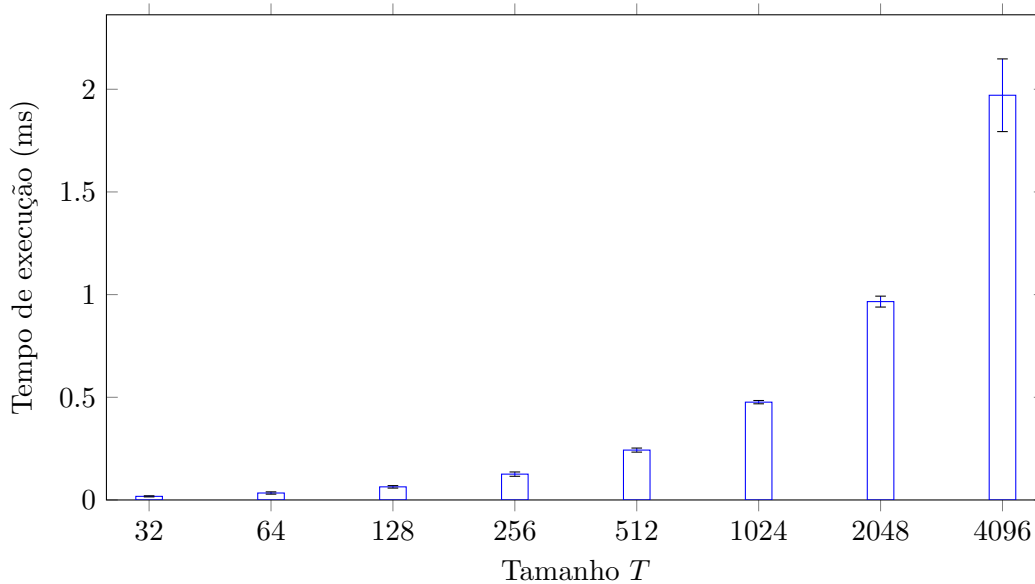


Figura 1: Tempo de execução média em função do tamanho T .

Isso fica mais evidente quando é mostrado o gráfico de linha sobre a vazão de vetor sobre unidade de tempo (ms) que diminui exponencialmente igual ao aumento de tempo levado para concluir a execução de um tamanho T .

Agora mostrando a vazão de elementos na figura 3, é possível notar que ela fica relativamente constante ao mudar de tamanho T e obviamente é o que deveria acontecer já que a máquina que está rodando os experimentos é a mesma, o acontecimento de a vazão de vetor diminuir é por causa do aumento T exponencial.

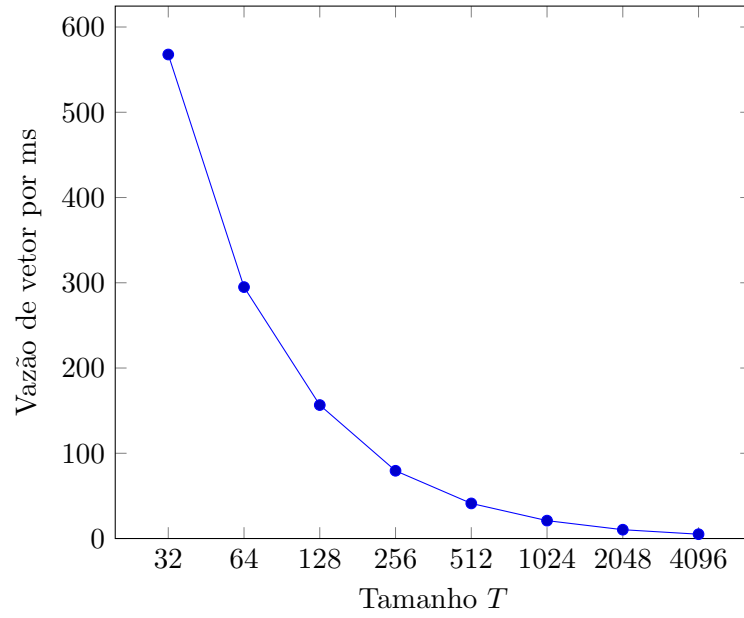


Figura 2: Vazão de vetor por unidade de tempo para cada tamanho T .

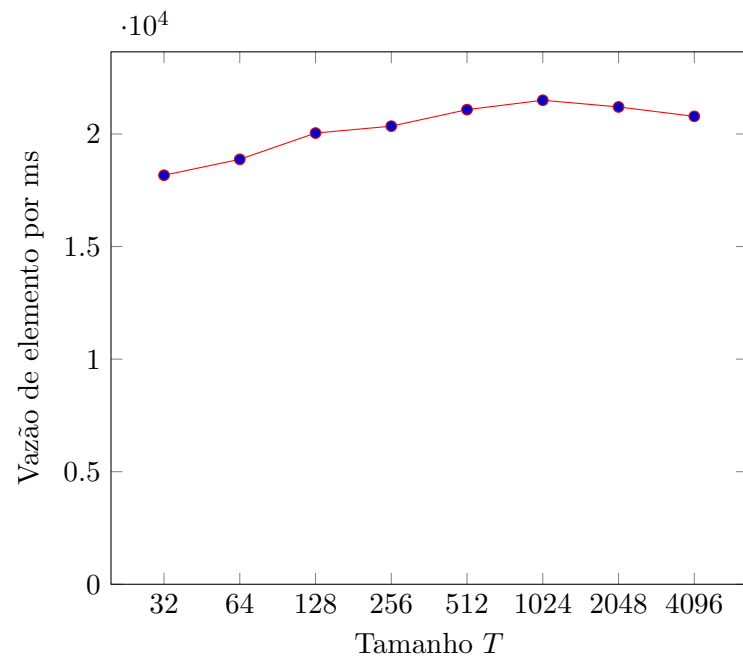


Figura 3: Vazão de elemento por unidade de tempo para cada tamanho T .

Referências

- [1] Igor Basilio. Data used in the article. https://docs.google.com/spreadsheets/d/1vleVur_CzIc3DqjRiVIEtTw7z5moGW5avWqr1f2B8vI/edit, 2026. Dataset supporting the results presented in the article. Accessed: 2026-08-20.
- [2] Igor Basilio. Distanciaeuclidiana. <https://github.com/Igor-Basilio/DistanciaEuclidiana>, 2026. Source code repository. Accessed: 2026-08-20.